Test Results & Quality Evidence

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Course: COEN838 — Advanced Software Engineering
Student: Idris Muhammad Abubakar
Matric No: P25EGCP9003
Date: October 2026

---

1. Executive Summary

Metric	Value
Total tests	**84**
Tests passing	**84 (100%)**
Fast tests (unit + service + integration)	79
System (E2E) tests	5
Line coverage	**96%** (473 statements, 19 missed)
SAST findings (Bandit)	**0**
CI status	✅ Green
Test framework	pytest 9.1.1 + httpx + Playwright

---

2. Testing Strategy (Module II)

ABU-SSO uses a **four-layer testing pyramid**, complemented by static analysis and continuous integration.

Layer	Test Count	Scope	Tooling
Unit	7	Pure functions (`hash_password`, `verify_password`)	pytest
Service	51	Business logic (`user_service`, `wallet_service`, `admin_service`)	pytest + SQLAlchemy
Integration	21	HTTP endpoints via FastAPI TestClient	pytest + httpx
System (E2E)	5	Full browser-driven user flows	pytest + Playwright + Chromium

Test isolation: every test runs against a shared **in-memory SQLite** database held open by `StaticPool`. All SQLAlchemy sessions (whether created via `app.database.SessionLocal` or the FastAPI `get_db`

dependency) connect to the **same in-memory database** — guaranteeing no cross-session drift. Tables are created once at import time and rows are wiped between tests via an autouse fixture.

Why StaticPool? A naive test setup with two separate engines silently produced "no such table: users" errors because in-memory SQLite creates a *new* database per connection. `StaticPool` keeps one connection alive for the lifetime of the engine, giving every session the same DB. This was a real bug we found and fixed (see commit `924b6e1`).

E2E isolation: the E2E layer spawns a **separate uvicorn subprocess on port 8001** with its own temp SQLite file, then drives it with **headless Chromium** via Playwright. Your local dev server (`:8000`) and `abu_sso.db` are untouched.

---

3. Coverage Report

Name Stmts Miss Cover Missing

app_init_.py 0 0 100% app\config.py 14 0 100% app\database.py 19 6 68% 14, 16, 39-43 app\main.py 19 0 100% app\models_init_.py 3 0 100% app\models\user.py 18 1 94% 26 app\models\wallet.py 27 2 93% 30, 46 app\routers_init_.py 0 0 100% app\routers\admin.py 33 2 94% 51-52 app\routers\auth.py 34 0 100% app\routers\health.py 5 0 100% app\routers\id_card.py 11 0 100% app\routers\pages.py 13 2 85% 15, 20, 25 app\routers\wallet.py 32 2 94% 55-56 app\schemas_init_.py 0 0 100% app\schemas\admin.py 29 0 100% app\schemas\id_card.py 8 0 100% app\schemas\user.py 24 0 100% app\schemas\wallet.py 22 0 100% app\services_init_.py 0 0 100% app\services\admin_service.py 38 0 100% app\services\qr_service.py 16 0 100% app\services\user_service.py 26 0 100% app\services\wallet_service.py 37 0 100% app\utils_init_.py 0 0 100% app\utils\jwt.py 31 1 97% 63 app\utils\rbac.py 9 0 100% app\utils\security.py 14 2 86% 22-23

TOTAL 473 19 96%

Target: ≥ 70% line coverage (rubric requirement).
Achieved: 96% — exceeds target by 26 percentage points.

Why some lines are uncovered: the remaining uncovered lines are defensive exception paths (e.g., SQLite file cleanup on session teardown, or environment-specific branches) that are difficult to exercise without environment manipulation. This is acceptable residual coverage.

Note on system tests: the E2E tests are marked with `@pytest.mark.e2e` and are **excluded from the default `pytest` run** for speed. They run explicitly with `pytest tests/e2e -m e2e` locally and are executed in a separate CI job on every push.

---

4. Test Inventory

4.1 Unit Tests — `tests/test_security.py` (7 tests)

Test	What it verifies
`test_hash_password_returns_string`	Returns a non-empty string
`test_hash_password_is_not_plaintext`	Password is never stored plaintext

Test	What it verifies
`test_hash_password_same_input_different_output`	Salt produces different hashes each time
`test_verify_password_correct`	Correct password verifies successfully
`test_verify_password_wrong`	Wrong password is rejected
`test_verify_password_empty_password`	Empty input is rejected
`test_hash_password_empty_raises`	Empty input raises `ValueError`

4.2 Service Tests

`tests/test_user_service.py` (16 tests) — CRUD operations, email normalisation, matric uppercasing, duplicate email, duplicate matric, password hashing, lookups by email / id / matric, case-insensitive matric, authentication success / failure / inactive-user rejection.

`tests/test_wallet_service.py` (12 tests) — Wallet auto-creation, idempotency, top-up, pay, insufficient funds, zero/negative amounts, transaction list ordering.

`tests/test_admin.py` (23 tests) — RBAC enforcement (401 without token, 403 for students), user listing, admin create-user (success, duplicate email, duplicate matric, non-admin blocked), role changes, activate/deactivate, deactivated user cannot log in, admin top-up, stats aggregation, service-level error paths.

4.3 Integration Tests — `tests/test_api_integration.py` (21 tests)

Full HTTP flows: register with matric, duplicate rejection, invalid input, matric login (correct, wrong, unknown, lowercase matric), missing/bad token, `/auth/me`, ID card generation, wallet top-up / pay / insufficient-funds / negative-amount, transaction listing, health check, root endpoint.

4.4 System (E2E) Tests — `tests/e2e/test_e2e.py` (5 tests)

Browser-driven tests that spin up a real server on port 8001 with a temporary database, launch headless Chromium, and simulate a user:

Test	What it verifies
`test_login_page_renders`	The `/login` page loads with matric + password fields
`test_login_success_redirects_to_dashboard`	Full authentication flow through the UI
`test_dashboard_shows_user_data_and_qr`	Profile, wallet, and QR code render correctly
`test_logout_redirects_to_login`	Session clearing + client-side guard
`test_student_cannot_access_admin_page`	RBAC enforced at UI level (admin button hidden, `/admin` redirects students)

5. Static Analysis (SAST)

Tool: Bandit 1.9.4
Command: `bandit -r app/`
Result: Test results: No issues identified.

Code scanned: Total lines of code: 525 Total lines skipped (#nosec): 0

Total issues (by severity): High: 0 Medium: 0 Low: 0

Interpretation: no common Python vulnerabilities (hardcoded credentials, unsafe `eval`, weak cryptography, shell injection, etc.) were detected.

---

6. Continuous Integration

Workflow: `.github/workflows/ci.yml`
Trigger: every push to `main` and every pull request

Job 1 — `test`:

1. Checkout code
2. Set up Python 3.12
3. Install dependencies from `requirements.txt`
4. Run `pytest -v` — the 79 fast tests must pass

Job 2 — `e2e` (runs only if Job 1 passes):

1. Checkout code + install dependencies
2. Install Playwright and Chromium (with system deps)
3. Run `pytest tests/e2e -m e2e -v` — the 5 browser-driven tests must pass

Status: ✅ Green
Average runtime: ~50 seconds (both jobs)

The CI badge in the README shows current status. A failing test blocks merge.

---

7. How to Reproduce These Results

Activate environment

conda activate abu-sso

Run fast tests (unit + service + integration)

pytest -q

Run E2E system tests

pytest tests/e2e -m e2e

Run coverage on fast tests

pytest --cov=app --cov-report=term-missing

Run with HTML coverage report

pytest --cov=app --cov-report=html

Open htmlcov/index.html in a browser

Run SAST

pip install bandit bandit -r app/ Expected outcome: 79 fast tests + 5 E2E tests pass, coverage ≥ 90%, Bandit reports 0 findings.

8. Traceability Every Functional Requirement (FR) is traced to at least one test. See docs/RTM.md for the full Requirements Traceability Matrix.

Coverage summary:

Category FRs covered % Authentication & registration (bootstrap) FR-01 to FR-06 100% ID card FR-07 100% Wallet FR-08 to FR-12 100% Roles & RBAC FR-13, FR-18, FR-19 100% Platform (health, UI) FR-14, FR-15 100% Admin create-user FR-16 100% Distributed SSO services FR-17 100% (manual + service-level) 9. Quality Metrics — Dashboard Metric Value Target Status Test pass rate 100% 100% ✅ Line coverage 96% ≥ 70% ✅ SAST findings 0 0 ✅ CI pass rate 100% 100% ✅ Tests per requirement ≥ 1 ≥ 1 ✅ Test layers 4 (unit + service + integration + system) ≥ 3 ✅ 10. Bugs Found and Fixed During Testing Test-driven development wasn't followed literally, but tests still caught real bugs. Two examples:

"no such table: users" in integration tests — caused by two separate SQLAlchemy engines pointing at different in-memory SQLite databases. Fixed by switching to a single engine with StaticPool (commit 924b6e1).

JWT secret too short — the placeholder change-me-in-production (23 bytes) triggered InsecureKeyLengthWarning. Fixed by extending the default and reading it from .env (RFC 7518 compliance).